

MathViz 3D — LLM Service Build Guide

Scope: This document covers the design and implementation guidance for the LLM Service — the intelligence layer of MathViz 3D. It manages conversation state, constructs prompts, calls LLM providers, produces Scene Description JSON, validates it, and forwards it to the Visualization Service. It is the only component that interacts with external AI APIs.

1. Purpose and Role

The LLM Service has one primary job: take a user's natural-language message and produce a well-formed, pedagogically sound `SceneDescription` JSON that the Visualization Service can execute.

It is responsible for:

- Managing multi-turn conversation history per session
- Building and maintaining the system prompt
- Calling the configured LLM provider (streaming)
- Extracting and validating the Scene Description from the LLM's response
- Triggering a self-correction retry if the Scene Description is malformed
- Forwarding the validated Scene Description to the Visualization Service
- Managing context window budget (pruning when needed)
- Storing conversation and scene history in Redis and PostgreSQL

It is explicitly **not** responsible for:

- Geometry computation
 - WebSocket streaming to the browser
 - User authentication (handled by the API Gateway)
 - Rendering or display logic
-

2. Service Structure

```
llm_service/  
├── main.py           # FastAPI app factory, lifespan, router registration  
├── config.py         # Settings via pydantic-settings  
└── dependencies.py   # FastAPI DI: Redis, DB, provider client
```

- └─ api/
 - └─ routes.py # POST /chat, GET /sessions/{id}/history
 - └─ models.py # Request/response Pydantic models for the HTTP API
- └─ provider/
 - └─ base.py # LLMProvider abstract base class
 - └─ anthropic.py # Anthropic Messages API implementation
 - └─ openai_compat.py # OpenAI-compatible API implementation (OpenAI, Azure, etc.)
 - └─ ollama.py # Local Ollama / vLLM implementation
 - └─ factory.py # Provider factory: reads config, returns correct impl
- └─ prompt/
 - └─ system_prompt.py # System prompt builder
 - └─ tool_definition.py # visualize_concept tool schema (mirrors SceneDescription)
 - └─ heuristics.py # Step design and narration heuristics injected into prompt
 - └─ templates/
 - └─ system_base.txt # Base system prompt template
 - └─ concept_types.txt # Concept type reference injected into prompt
 - └─ expression_syntax.txt # Expression syntax rules injected into prompt
- └─ conversation/
 - └─ manager.py # Conversation lifecycle: create, append, prune, retrieve
 - └─ context_window.py # Token budget management and sliding-window pruning
 - └─ models.py # Message, ConversationState Pydantic models
- └─ extraction/
 - └─ scene_extractor.py # Extract Scene Description from LLM response (tool call or JSON)
 - └─ validator.py # Multi-stage Scene Description validation pipeline
 - └─ repairer.py # Auto-repair for trivially fixable validation failures
 - └─ retry.py # Self-correction retry orchestration
- └─ forwarding/
 - └─ viz_client.py # HTTP client to POST Scene Description to Visualization Service
- └─ storage/
 - └─ redis_store.py # Conversation context, scene history in Redis
 - └─ db_store.py # Persist conversations and scenes to PostgreSQL
- └─ tests/
 - └─ test_prompt.py # System prompt construction tests
 - └─ test_extraction.py # Scene Description extraction from mock LLM responses
 - └─ test_validator.py # Validation pipeline tests
 - └─ test_repairer.py # Auto-repair tests

```
└─ test_conversation.py    # Context window pruning and history retrieval tests
└─ test_integration.py     # Full turn: message in → Scene Description forwarded
```

3. Configuration (`config.py`)

All settings via `pydantic-settings` from environment variables.

Provider settings: `LLM_PROVIDER` (anthropic | openai | ollama), `LLM_API_KEY`, `LLM_MODEL` (default: `claude-sonnet-4-20250514` for Anthropic), `LLM_BASE_URL` (for OpenAI-compatible or Ollama), `LLM_MAX_TOKENS` (default 4096 for Scene Description output), `LLM_TEMPERATURE` (default 0.2 — low for structured output), `LLM_STREAMING` (default true).

Context window: `CONTEXT_TOKEN_BUDGET` (default 32,000 tokens), `CONTEXT_RESERVE_FOR_OUTPUT` (default 4,096 tokens — always reserved for the response), `MIN_TURNS_TO_RETAIN` (default 4 — never prune below this many turns regardless of budget).

Retry: `MAX_SELF_CORRECTION_RETRIES` (default 1), `RETRY_INCLUDE_ERROR_DETAILS` (default true — include validation error in the retry prompt).

Forwarding: `VIZ_SERVICE_URL`, `VIZ_SERVICE_INTERNAL_SECRET` (shared HMAC secret for service-to-service auth), `VIZ_FORWARD_TIMEOUT_S` (default 10).

Redis and PostgreSQL: same pattern as Visualization Service.

4. API Layer (`api/`)

4.1 Routes (`routes.py`)

`POST /chat`

The primary endpoint. Accepts a user message and session context, runs the full LLM turn pipeline, and returns the assistant's response plus the forwarded scene ID (if a visualization was produced).

Request body:

- `session_id: UUID`
- `user_id: UUID`
- `message: str` (the user's natural-language input)
- `stream_response: bool` (default true — stream the assistant's text back to the client)

Response (non-streaming): `{"assistant_text": str, "scene_id": UUID | null, "turn_id": UUID}`

Response (streaming): Server-Sent Events stream. Events:

- `{event: text_delta, data: {"delta": str}}` — partial assistant text
- `{event: scene_ready, data: {"scene_id": UUID}}` — emitted when Scene Description is validated and forwarded
- `{event: done, data: {}}` — end of turn

`GET /sessions/{session_id}/history`

Returns the conversation history for a session: list of `{role, content, turn_id, created_at, scene_id}` dicts. Used by the frontend to restore the chat panel after reconnection.

`DELETE /sessions/{session_id}`

Clears session state from Redis (conversation history, scene context). PostgreSQL records are retained for analytics.

4.2 Request/Response Models (`models.py`)

`ChatRequest`: `session_id`, `user_id`, `message` (max 4,000 chars, validated), `stream_response`.

`ChatResponse`: `assistant_text`, `scene_id` (nullable), `turn_id`, `latency_ms`.

`HistoryItem`: `role` (user | assistant), `content`, `turn_id`, `created_at`, `scene_id` (nullable — links an assistant turn to the scene it produced).

5. LLM Provider Layer (`provider/`)

5.1 Base Class (`base.py`)

```
python
```

```

class LLMProvider(ABC):
    @abstractmethod
    async def complete(
        self,
        messages: list[Message],
        system_prompt: str,
        tools: list[ToolDefinition],
        max_tokens: int,
        temperature: float,
    ) -> LLMResponse:
        ...

    @abstractmethod
    async def stream_complete(
        self,
        messages: list[Message],
        system_prompt: str,
        tools: list[ToolDefinition],
        max_tokens: int,
        temperature: float,
    ) -> AsyncIterator[LLMStreamEvent]:
        ...

```

`LLMResponse` is a dataclass: `text_content: str`, `tool_calls: list[ToolCall]`, `input_tokens: int`, `output_tokens: int`, `stop_reason: str`.

`LLMStreamEvent` is a union type: `TextDelta(delta: str)` | `ToolCallDelta(index: int, delta: str)` | `Done(response: LLMResponse)`.

`ToolCall` is a dataclass: `tool_name: str`, `tool_input: dict` (the raw parsed JSON from the model's tool use block).

`ToolDefinition` is a dataclass: `name: str`, `description: str`, `input_schema: dict` (JSON Schema).

5.2 Anthropic Implementation (`anthropic.py`)

Uses the Anthropic Python SDK (async client). Maps the internal `Message` format to Anthropic's `messages` array format. The system prompt is passed as the `system` parameter (not as a message).

Tool use: passes the `visualize_concept` tool definition in the `tools` parameter. Detects tool use blocks in the response (`content` blocks with `type == "tool_use"`) and maps them to `ToolCall` objects.

Streaming: uses `client.messages.stream()` context manager. Emits `TextDelta` events for `text_delta` content block events and `ToolCallDelta` events for `input_json_delta` events. Accumulates the full tool call JSON from deltas and parses it at the end of the stream.

Token counting: reads `usage.input_tokens` and `usage.output_tokens` from the final stream event.

Error handling: wrap SDK exceptions in a `LLMProviderError` with the provider name, status code, and message. Retry on 529 (overloaded) with exponential backoff up to 3 attempts. Raise immediately on 400 (bad request — likely a prompt issue) and 401 (auth failure).

5.3 OpenAI-Compatible Implementation (`openai_compat.py`)

Uses the `openai` Python SDK (async client) with a configurable `base_url`. This handles OpenAI, Azure OpenAI, and any vLLM endpoint that exposes the OpenAI Chat Completions API.

Tool use: maps to OpenAI's `tools` parameter with `function` type. Detects `tool_calls` in the response choices and maps to `ToolCall` objects.

Streaming: uses `client.chat.completions.stream()`. Similar delta accumulation pattern to Anthropic.

5.4 Ollama Implementation (`ollama.py`)

Uses `httpx` async client directly (no official Python SDK for Ollama). Calls `/api/chat` endpoint with `stream: true`. Ollama does not natively support tool use for all models — fall back to JSON-mode prompting (see Section 7.2) when tools are not supported.

Detect tool support by checking the model name against a configurable allowlist. For unsupported models, instruct the LLM to output a JSON block and extract it via the JSON extraction path in `scene_extractor.py`.

5.5 Factory (`factory.py`)

Reads `LLM_PROVIDER` from config and returns the correct provider instance. Singleton — instantiated once at app startup and injected via FastAPI dependency.

6. Prompt Layer (`prompt/`)

6.1 System Prompt Builder (`system_prompt.py`)

The system prompt is assembled dynamically at startup from template files and injected heuristics. It is a single string passed to the provider on every call. It does not change between turns (conversation history handles context evolution).

Assembly order:

1. **Role framing** (from `system_base.txt`): establishes the LLM as a mathematical pedagogy expert whose special ability is producing `SceneDescription` JSON via the `visualize_concept` tool.
2. **Tool capability statement**: explains that when the user asks to visualize, explore, or show a mathematical concept, the LLM must call `visualize_concept` with a complete Scene Description. For conversational or

follow-up questions that do not require a new visualization, the LLM responds with plain text only.

3. **Concept type reference** (from `concept_types.txt`): a compact reference listing every registered `ConceptType` with a one-line description and a minimal example expression. This is the LLM's menu of what it can visualize.
4. **Expression syntax rules** (from `expression_syntax.txt`): rules for the `expression` field — valid operators, function names, variable names, how to write parametric expressions as lists, how to write matrix expressions.
5. **Step design heuristics** (from `heuristics.py`): injected as a numbered list:
 - Each step introduces exactly one new idea.
 - Simple concepts: 3–5 steps. Complex concepts: up to 12 steps.
 - Step 1 always establishes the baseline (the simplest form of the concept).
 - The final step always shows the complete or converged result.
 - `layer_mode` should be `add` for cumulative concepts (Fourier, Riemann), `replace` for transformations, `highlight` when comparing new to old.
 - Narration sentences are written for an undergraduate audience and connect the visual change to the mathematical meaning.
6. **HUD label guidelines**: `hud_equation` must be the simplified, human-readable LaTeX form — not the raw evaluation string. Example: use `S_3 = \sin(x) + \frac{\sin(3x)}{3}`, not `sin(x)+sin(3*x)/3`.
7. **Hard prohibitions**: the LLM must never include rendering code, shader terminology, Three.js-specific language, or geometry algorithms in its Scene Description output.

The assembled prompt is cached as a module-level string after first construction. If the concept type registry changes (e.g., new types added), the service must be restarted to pick up the change.

6.2 Tool Definition (`tool_definition.py`)

The `visualize_concept` tool is defined as a JSON Schema object that exactly mirrors the `SceneDescription` Pydantic model. This is the single source of truth for what the LLM is allowed to produce.

Key design decisions:

- All required fields are marked `required` in the schema. The LLM cannot omit them.
- Enum fields list all valid values explicitly. The LLM cannot hallucinate new concept types or layer modes.
- `expression` uses `oneOf: [string, array of strings]` to allow both single and multi-expression steps.
- `hud_equation` has `maxLength: 500` and `minLength: 1` — validated at the schema level before even reaching Python validation.

- Nested objects (`DomainSpec`, `RenderHints`, `Annotation`) are defined as `$defs` and referenced, keeping the schema readable.
- `total_steps` and `len(steps)` cannot be validated at the JSON Schema level — this is caught by the Pydantic validator downstream.

The tool schema is serialized to a dict at module load time and passed to the provider on every call.

6.3 Heuristics (`heuristics.py`)

A Python module (not a template file) that generates the heuristics section of the system prompt programmatically. This allows heuristics to reference the actual registered concept types (pulled from the handler registry at import time) rather than being hardcoded in a text file.

Example dynamic generation: "For `riemann_sum` concepts, always begin with n=4 subdivisions in Step 1 and increase by doubling each step. The `parameters` field must include `n` with the subdivision count for each step."

Heuristics are grouped by: general step design, layer mode selection, narration style, concept-type-specific rules, and common mistakes to avoid.

7. Conversation Manager (`conversation/`)

7.1 Manager (`manager.py`)

Manages the full lifecycle of a conversation session.

`async create_session(user_id, session_id) -> ConversationState` Initializes a new conversation state in Redis. Returns the initial `ConversationState`.

`async append_turn(session_id, role, content, scene_id=None) -> None` Appends a new message to the conversation history in Redis. If the new history exceeds the token budget, calls `context_window.prune()` before saving.

`async get_messages_for_prompt(session_id) -> list[Message]` Returns the conversation history formatted for the LLM provider. Applies pruning if needed. Always includes the last Scene Description in full (see Section 7.2).

`async get_state(session_id) -> ConversationState` Returns the current conversation state including concept history, last scene ID, and token usage.

7.2 Context Window Management (`context_window.py`)

The context window has a fixed budget (default 32k tokens). The system prompt and tool definition consume a fixed portion. The remaining budget is split between conversation history and the output reservation.

Token counting: use the provider's token counting API when available (Anthropic provides `count_tokens`); for others, use `tiktoken` with the appropriate encoding as an approximation). Count tokens for each message at append time and cache the count in the message object.

Pruning strategy (sliding window with pinning):

The following are always pinned (never pruned):

- The most recent `MIN_TURNS_TO_RETAIN` turns (default 4, meaning 4 user + 4 assistant messages = 8 messages).
- The most recent complete Scene Description JSON (the last assistant turn that included a `visualize_concept` tool call). This is pinned because the LLM needs it to answer follow-up questions like "add the derivative to step 3."
- Any user turn that references a specific scene by name ("go back to the Fourier example") — pinned until a new scene with that name is created.

Pruning removes the oldest non-pinned turns first, one at a time, until the total token count is within budget.

Concept history: a separate lightweight list stored in `ConversationState`, tracking `{concept_title, scene_id, step_count, turn_index}` for every scene produced in the session. This is never pruned and is appended to the system prompt as a compact reference (e.g., "Previous visualizations this session: [1] Fourier Series Convergence (5 steps), [2] Riemann Sum for sin(x) (6 steps)"). This allows the LLM to reference prior concepts even after their full conversation turns have been pruned.

7.3 Message Model (`models.py`)

`Message` dataclass:

- `role: Literal["user", "assistant", "tool_result"]`
- `content: str | list[ContentBlock]` — plain text for user/assistant; list of content blocks for tool use
- `turn_id: UUID`
- `token_count: int` (cached at append time)
- `scene_id: UUID | None` — set if this turn produced a scene
- `created_at: datetime`

`ConversationState` dataclass:

- `session_id: UUID`
- `user_id: UUID`
- `messages: list[Message]`
- `concept_history: list[ConceptHistoryItem]`

- `last_scene_id: UUID | None`
 - `total_input_tokens: int` (running total for usage tracking)
 - `total_output_tokens: int`
-

8. Scene Description Extraction (`extraction/`)

8.1 Extractor (`scene_extractor.py`)

Entry point: `extract_scene_description(llm_response: LLMResponse) -> dict | None`

The LLM can produce a Scene Description in two ways:

1. **Tool call** (preferred): the response contains a `ToolCall` with `tool_name == "visualize_concept"`. The `tool_input` dict is already parsed JSON — use it directly.
2. **JSON block** (fallback for providers that don't support tool use): scan the text content for a JSON block delimited by ````json ... ```` or `<scene_description>...</scene_description>` tags. Extract and parse the JSON with `json.loads()`.

If neither is found, return `None` — this is a conversational response with no visualization. The assistant text is still returned to the user.

If both are found (malformed output), prefer the tool call.

Error handling: if `json.loads()` fails on an extracted block, raise `ExtractionError` with the raw string. Do not attempt to repair JSON at this stage — that is the validator's job.

8.2 Validation Pipeline (`validator.py`)

Entry point: `validate_scene_description(raw_dict: dict) -> SceneDescription`

Runs the multi-stage pipeline. Each stage either passes, auto-repairs (via `repairer.py`), or raises `ValidationError` with structured details.

Stage 1 — Pydantic schema validation: `SceneDescription.model_validate(raw_dict)`. Catches type errors, missing fields, enum violations. On failure: pass the error to `repairer.py` for trivial repairs (e.g., wrong case on an enum value: `"Fade_In"` → `"fade_in"`). If repair succeeds, continue. If not, raise.

Stage 2 — Expression parse check: For every `expression` field across all steps, call `expression/parser.py`'s `parse_expression()`. Collect all failures. If any expression fails to parse, raise `ValidationError` listing all failed steps and expressions. Do not continue to Stage 3 if expressions are broken — the Rust core cannot handle unparseable ASTs.

Stage 3 — Step coherence check:

- Verify `step_index` values are contiguous starting from 1 with no gaps.
- Verify `total_steps == len(steps)`.
- Check that `layer_mode` transitions are semantically valid: a `highlight` on Step 1 is a warning (there is nothing to highlight); a `replace` after a long `add` sequence is valid but worth logging.
- Check that animated parameters (`{min, max, frames}`) have `frames >= 2`.

Stage 4 — Domain sanity check: For every `DomainSpec` (global and per-step overrides): assert `min < max` for every axis. Clamp `steps` to `[64, 2048]` silently (log the clamp). This is a silent repair, not a failure.

Stage 5 — Concept type check: Assert every `concept_type` value is in the registered handler registry. If not, raise `ValidationError` naming the unsupported type.

All stages that pass return the (potentially auto-repaired) `SceneDescription` model. The final validated model is what gets forwarded to the Visualization Service.

8.3 Auto-Repairer (`repairer.py`)

Handles trivial, safe repairs without involving the LLM. Repairs are only applied if they do not change the semantic meaning of the Scene Description.

Repair catalogue:

- **Enum case normalization:** lowercase all `concept_type`, `layer_mode`, `transition`, `coordinate_system` values.
- **Missing `version` field:** default to `"2.0"`.
- **Missing `default_camera`:** omit the field (it is optional).
- **`total_steps` mismatch:** if `total_steps` is off by one and the step list is otherwise valid, correct `total_steps` to match `len(steps)`.
- **Contiguous step index off-by-one:** if all steps are present but `step_index` starts at 0 instead of 1, increment all indices by 1.
- **`hud_equation` missing on a step:** copy the `expression` field (as a string) as the `hud_equation` fallback.
- **Domain `steps` out of bounds:** clamp to `[64, 2048]`.
- **Empty `narration`:** substitute a generic narration string like `"Step N."` — valid but logged as a quality warning.

Repairs that are not in this catalogue are not attempted. Anything more complex goes to the self-correction retry.

8.4 Self-Correction Retry (`retry.py`)

Entry point: `async run_with_retry(llm_turn_fn, session_id, max_retries) -> SceneDescription`

If the validation pipeline raises after auto-repair, the retry orchestrator:

1. Appends the failed Scene Description and a structured error message to the conversation as a `tool_result` message (for providers that support it) or as a user-role correction message.
2. Constructs a correction prompt: "The Scene Description you produced has the following errors: [structured error list]. Please correct only the listed issues and re-emit the full corrected Scene Description."
3. Calls the LLM provider again with the updated conversation history. The retry call uses the same streaming path as the original.
4. Extracts and validates the new response. If it passes, use it. If it fails again (and `max_retries > 1`), repeat up to the configured limit.
5. If all retries are exhausted, raise `SceneDescriptionExhaustedError`. The caller catches this, sends the user a message explaining that the visualization could not be generated, and does not forward anything to the Visualization Service.

The retry turn is included in conversation history — this means the LLM in subsequent turns can see what went wrong and avoid repeating the mistake.

9. Full Turn Orchestration

The core logic of a single `/chat` turn, combining all the above components:

9.1 Turn Flow

1. Receive ChatRequest (session_id, user_id, message)
2. Load ConversationState from Redis
3. Append user message to history (with token count)
4. Prune context window if over budget
5. Build prompt:
 - a. system_prompt (static, cached)
 - b. messages (from conversation manager)
 - c. tools: [visualize_concept tool definition]
6. Call LLM provider (streaming):
 - a. Stream TextDelta events → forward to client via SSE
 - b. Accumulate ToolCallDelta events → buffer tool call JSON
7. On stream completion:
 - a. If tool call present:
 - i. Extract raw dict
 - ii. Run validation pipeline (with auto-repair)

- iii. If validation fails: run self-correction retry (go to step 6 with correction)
- iv. Forward validated SceneDescription to Visualization Service
- v. Receive scene_id from Visualization Service
- vi. Emit 'scene_ready' SSE event to client
- b. If no tool call: plain conversational response, no forwarding
- 8. Append assistant turn to conversation history
- 9. Update ConversationState (concept history, last_scene_id, token usage)
- 10. Persist turn to PostgreSQL (async, non-blocking)
- 11. Emit 'done' SSE event

9.2 Follow-Up Query Handling

When the user sends a follow-up referencing a prior scene ("now add the derivative to step 3", "make the surface denser", "go back to the Fourier example"):

- The conversation history contains the prior Scene Description (pinned in context).
- The LLM can read the prior scene and produce a new Scene Description that modifies it.
- The new Scene Description is treated identically to a fresh one — fully validated and forwarded.
- The old scene's step bundles in Redis remain valid and are not invalidated. The new `scene_id` is distinct.

The LLM Service does not parse or interpret follow-up intent itself — it relies entirely on the LLM's in-context understanding of the conversation history.

10. Visualization Service Forwarding (`forwarding/`)

10.1 Client (`viz_client.py`)

A thin async HTTP client (using `httpx.AsyncClient`) that forwards the validated Scene Description to the Visualization Service.

`async forward_scene(scene: SceneDescription, session_id: UUID, user_id: UUID) -> ForwardResult`

1. Serialize the `SceneDescription` to JSON.
2. POST to `{VIZ_SERVICE_URL}/scenes` with:
 - Body: `{"scene_description": scene_json, "session_id": ..., "user_id": ...}`
 - Header: `X-Internal-Secret: {VIZ_SERVICE_INTERNAL_SECRET}` (HMAC-based service-to-service auth)
3. Expect a 201 response with `{"scene_id": UUID, "ws_url": str}`.
4. Return `ForwardResult(scene_id, ws_url)`.

Error handling:

- Timeout (default 10 s): raise `ForwardingTimeoutError`. Inform the user the visualization service is unavailable.
 - 4xx from Visualization Service (re-validation failure): raise `ForwardingRejectedError` with the response body. This means the scene passed LLM Service validation but failed re-validation in the Visualization Service — a schema drift bug. Log at ERROR level for investigation.
 - 5xx: retry once after 1 s, then raise `ForwardingUnavailableError`.
-

11. Storage Layer (`storage/`)

11.1 Redis Store (`redis_store.py`)

Conversation context: stored as a JSON-serialized list of `Message` objects at key `llm_ctx:{session_id}`, TTL 4 hours. Updated on every turn.

Conversation state: the full `ConversationState` (including concept history and token counters) stored as a JSON hash at `llm_session:{session_id}`, TTL 4 hours.

Rate limit counters: `rl_llm:{user_id}:{window}` — incremented on each LLM call. Windows: per-minute and per-day. Checked before calling the provider; reject with HTTP 429 if exceeded.

11.2 DB Store (`db_store.py`)

Persists to PostgreSQL via SQLAlchemy 2.0 async.

On each turn (async, fire-and-forget):

- Insert into `messages`: `(conversation_id, role, content, scene_description_json, created_at)`.
- Update `conversations`: `(last_scene_id, last_active_at)`.
- Insert into `usage_events`: `(user_id, "llm_turn", {input_tokens, output_tokens, provider, model})`.

On scene forward success:

- Insert into `scene_descriptions`: `(user_id, scene_json, title, created_at)`.
- Insert into `step_states` for each step (geometry cache key is populated later by the Visualization Service via a callback or polling).

Persistence failures are logged but do not fail the turn — the Redis copy is the operational source of truth. PostgreSQL is for analytics, user history, and long-term storage.

12. Testing Strategy

12.1 Unit Tests

Prompt tests (`test_prompt.py`):

- System prompt contains all registered concept type names.
- Tool definition JSON Schema validates against the `SceneDescription` Pydantic model (no drift between schema and model).
- Heuristics section is non-empty and contains at least one rule per concept type category.

Extraction tests (`test_extraction.py`):

- Tool call response → correct dict extracted.
- JSON block in text → correct dict extracted.
- Neither present → `None` returned.
- Malformed JSON block → `ExtractionError` raised.
- Both tool call and JSON block → tool call wins.

Validation tests (`test_validator.py`):

- Valid Scene Description (from fixture) passes all stages.
- Missing required field → Stage 1 failure.
- Invalid expression → Stage 2 failure naming the offending step.
- Non-contiguous step indices → Stage 3 failure.
- Unknown concept type → Stage 5 failure.
- Trivially repairable errors (enum case, `total_steps` off by one) → repaired silently, validation passes.

Retry tests (`test_repairer.py`):

- Each repair in the catalogue is tested: input the broken dict, assert the repaired output is valid.
- Non-repairable error is not silently swallowed — `ValidationError` is re-raised.

Conversation tests (`test_conversation.py`):

- Pruning removes oldest non-pinned messages when over budget.
- Last Scene Description is never pruned.
- `MIN_TURNS_TO_RETAIN` is always respected.

- Concept history is maintained across pruning.

12.2 Integration Tests (`test_integration.py`)

Use a mock LLM provider that returns pre-written responses (fixture files). Use a mock Visualization Service that accepts `POST /scenes` and returns a canned `scene_id`.

Key scenarios:

- Valid request → Scene Description extracted, validated, forwarded, `scene_ready` SSE event emitted.
- LLM returns invalid Scene Description → auto-repair applied, validation passes, forwarded.
- LLM returns non-repairable Scene Description → retry triggered, mock provider returns corrected response on second call, forwarded successfully.
- All retries exhausted → `SceneDescriptionExhaustedError`, user receives error message, nothing forwarded.
- Conversational response (no tool call) → no forwarding, assistant text streamed to client.
- Follow-up message referencing prior scene → prior scene JSON present in messages sent to mock provider, new scene forwarded with distinct `scene_id`.

12.3 Provider Contract Tests

One test file per provider implementation. Uses recorded HTTP fixtures (via `pytest-recording` or `respx`) rather than live API calls.

Tests assert:

- Messages are formatted correctly for the provider's API.
- Tool definitions are passed in the expected format.
- Tool call responses are parsed into `ToolCall` objects correctly.
- Streaming deltas are reassembled into the correct final response.
- Token counts are extracted from the response correctly.

13. Security

- **LLM API keys** are read from environment variables at startup and never logged, never included in error messages, and never serialized to Redis or PostgreSQL.
- **User message content** is length-limited to 4,000 characters at the API layer before reaching the conversation manager or the LLM. Oversized inputs are rejected with HTTP 400.

- **Prompt injection defense:** the system prompt includes an explicit instruction that any content in user messages that attempts to override system instructions, change the LLM's role, or produce non-JSON output in the `visualize_concept` tool must be ignored. This is belt-and-suspenders — the Pydantic validation pipeline will catch any malformed output regardless.
 - **Rate limiting** at two levels: the API Gateway enforces per-session request rate limits; the LLM Service enforces per-user LLM call limits (separate from the gateway, to protect provider API quota).
 - **Service-to-service auth** via a shared HMAC secret (not user-facing JWT). The Visualization Service rejects any `POST /scenes` that lacks a valid `X-Internal-Secret` header.
 - **No user input reaches the Rust core.** The only path from user content to computation is: user message → LLM → Scene Description JSON → Pydantic validation → SymPy parsing → AST dict → Rust. Every stage validates independently.
-

14. Extension Points

Adding a new LLM provider:

1. Implement `LLMProvider` in a new file under `provider/`.
2. Register it in `provider/factory.py`.
3. Add provider contract tests.
4. No other changes required.

Adding a new concept type to the prompt:

1. Add the type to `concept_types.txt` and `heuristics.py`.
2. Add a usage example to the tool definition description.
3. Restart the service to pick up the new system prompt.
4. (The handler and Rust implementation are registered separately in the Visualization Service.)

Changing the Scene Description schema:

1. Update the Pydantic model in `schema/scene_description.py` (shared or mirrored from the Visualization Service).
2. Update the tool definition JSON Schema in `prompt/tool_definition.py` to match.
3. Update the repairer if new trivially-repairable cases arise.
4. Bump the `version` field to the new protocol version.
5. Run the provider contract tests — mismatches between the tool schema and model will surface here.

